# Biconoirs Gourmet - Backend Business Rules

Postman testing guide for all current URIs of the NestJS + Prisma + Supabase project.

## Base Configuration

```
Recommended Postman variable:
baseUrl = http://localhost:3000/ops

When deploying on AWS:
baseUrl = http://PUBLIC_IP_EC2:3000/ops
# or with a domain:
baseUrl = https://YOUR_DOMAIN/ops
```

Common headers:

```
Content-Type: application/json

# For cart and Auth /me:
x-user-id: {{userId}}

# For inventory:
x-user-id: {{adminUserId}}

# For logout:
authorization: {{refreshToken}}
```

Suggested variables in Postman:

```
baseUrl
userId
adminUserId
customerId
dishId
categoryId
orderId
inventoryId
skuCode
refreshToken
resetToken
```

## A - Ingredientes

### GET /ops/ingredients/:ingredientId

Get a specific ingredient by ID.

| | |
|---|---|
| **Method** | `GET` |
| **URL** | `{{baseUrl_ops}}/ingredients/:ingredientId` |

Body: without body.

### POST /ops/ingredients

Create a new ingredient.

| Method | POST |
| --- | --- |
| URL | {{baseUrl_ops}}/ingredients |

Body JSON:

```
{
  "name": "Tomato",
  "category": "vegetables",
  "unit": "kg",
  "stock": 20
}
```

### PUT/PATCH /ops/ingredients/:ingredientId

Update an existing ingredient (full or partial).

| Method | PUT/PATCH |
| --- | --- |
| URL | {{baseUrl_ops}}/ingredients/:ingredientId |

Body JSON:

```
{
  "name": "Tomato Updated",
  "stock": 35
}
```

### DELETE /ops/ingredients/:ingredientId

Remove an ingredient by ID.

| Method | DELETE |
| --- | --- |
| URL | {{baseUrl_ops}}/ingredients/:ingredientId |

Body: without body.

## A - Encuestas

### GET /ops/surveys

List all surveys.

| Method | GET |
| --- | --- |
| URL | {{baseUrl_ops}}/surveys |

Body: without body.

### GET /ops/surveys/:surveyId

Get a specific survey by ID.

| Method | GET |
| --- | --- |
| URL | {{baseUrl_ops}}/surveys/:surveyId |

Body: without body.

### POST /ops/surveys/:surveyId/submit

Submit a survey response.

| Method | POST |
| --- | --- |
| URL | {{baseUrl_ops}}/surveys/:surveyId/submit |

Body JSON:

```
{
  "responses": [
    { "questionId": "q1", "answer": "Very satisfied" },
    { "questionId": "q2", "answer": "5" }
  ]
}
```

## B - Auth

### POST /api/v1/auth/signup

Register a new user.

| Method | POST |
| --- | --- |
| URL | {{baseUrl_api}}/auth/signup |

Body JSON:

```
{
  "name": "Maria Lopez",
  "email": "maria@test.com",
  "password": "securePass123",
  "phone": "0999999999"
}
```

### POST /api/v1/auth/login

Authenticate user and obtain access token.

| Method | POST |
| --- | --- |
| URL | {{baseUrl_api}}/auth/login |

Body JSON:

```
{
  "email": "maria@test.com",
  "password": "securePass123"
}
```

### POST /api/v1/auth/logout

Revoke the current session token.

| Method | POST |
| --- | --- |
| URL | {{baseUrl_api}}/auth/logout |

Headers:

```
Authorization: Bearer {{accessToken}}
```

Body: without body.

### POST /api/v1/auth/password-recovery

Send a password recovery email.

| Method | POST |
| --- | --- |
| URL | {{baseUrl_api}}/auth/password-recovery |

Body JSON:

```
{
  "email": "maria@test.com"
}
```

## B - Perfil

### GET /api/v1/profile

Get the current user's profile.

| Method | GET |
| --- | --- |
| URL | {{baseUrl_api}}/profile |

Headers:

```
Authorization: Bearer {{accessToken}}
```

Body: without body.

### PUT/PATCH /api/v1/profile

Update the current user's profile (full or partial).

| Method | PUT/PATCH |
| --- | --- |
| URL | {{baseUrl_api}}/profile |

Headers:

```
Authorization: Bearer {{accessToken}}
```

Body JSON:

```
{
  "name": "Maria Lopez Updated",
  "phone": "0988888888"
}
```

# B - Menú

## GET /api/v1/menu

List all available dishes in the menu.

| Method | GET |
| --- | --- |
| URL | {{baseUrl_api}}/menu |

Body: without body.

## GET /api/v1/menu/:dishId

Get details of a specific dish.

| Method | GET |
| --- | --- |
| URL | {{baseUrl_api}}/menu/:dishId |

Body: without body.

# B - Carrito

## POST /api/v1/cart/add

Add a dish to the user's cart.

| Method | POST |
| --- | --- |
| URL | {{baseUrl_api}}/cart/add |

Headers:

```
Authorization: Bearer {{accessToken}}
```

Body JSON:

```
{
```

```
  "dishId": "{{dishId}}",
  "quantity": 2
}
```

### GET /api/v1/cart

View the current user's cart.

| Method | GET |
| --- | --- |
| URL | {{baseUrl_api}}/cart |

Headers:

```
Authorization: Bearer {{accessToken}}
```

Body: without body.

## B - Checkout

### POST /api/v1/checkout

Process the checkout and create an order from the cart.

| Method | POST |
| --- | --- |
| URL | {{baseUrl_api}}/checkout |

Headers:

```
Authorization: Bearer {{accessToken}}
```

Body JSON:

```
{
  "deliveryType": "delivery",
  "deliveryAddress": "Av. Principal 123",
  "specialInstructions": "No onion",
  "paymentMethod": "card"
}
```

## B - Órdenes

### GET /api/v1/orders

List all orders belonging to the authenticated user.

| Method | GET |
| --- | --- |
| URL | {{baseUrl_api}}/orders |

Headers:

```
Authorization: Bearer {{accessToken}}
```

Body: without body.

# B - Reservas

## POST /api/v1/reservations

Create a new table reservation.

| Method | POST |
| --- | --- |
| URL | {{baseUrl_api}}/reservations |

Headers:

```
Authorization: Bearer {{accessToken}}
```

Body JSON:

```
{
  "date": "2026-07-20",
  "time": "19:30",
  "guests": 4,
  "notes": "Window table preferred"
}
```

## GET /api/v1/reservations

List all reservations for the authenticated user.

| Method | GET |
| --- | --- |
| URL | {{baseUrl_api}}/reservations |

Headers:

```
Authorization: Bearer {{accessToken}}
```

Body: without body.

## PUT/PATCH/DELETE /api/v1/reservations/:reservationId

Update or cancel a specific reservation.

| Method | PUT/PATCH/DELETE |
| --- | --- |
| URL | {{baseUrl_api}}/reservations/:reservationId |

Headers:

```
Authorization: Bearer {{accessToken}}
```

Body JSON (for PUT/PATCH):

```
{
  "date": "2026-07-21",
```

```
  "time": "20:00",
  "guests": 2
}
```

Body: without body (for DELETE).

# B - Encuestas

## GET /api/v1/surveys/active

Get the currently active survey available to users.

| Method | GET |
| --- | --- |
| URL | {{baseUrl_api}}/surveys/active |

Headers:

```
Authorization: Bearer {{accessToken}}
```

Body: without body.

## POST /api/v1/surveys/:surveyId/respond

Submit a response to a specific survey.

| Method | POST |
| --- | --- |
| URL | {{baseUrl_api}}/surveys/:surveyId/respond |

Headers:

```
Authorization: Bearer {{accessToken}}
```

Body JSON:

```
{
  "answers": [
    { "questionId": "q1", "value": "Excellent" },
    { "questionId": "q2", "value": "5" }
  ]
}
```

# B - Admin

## GET/POST/PUT/DELETE /api/v1/admin/inventory

Full CRUD for inventory management. Requires admin role.

| Method | GET/POST/PUT/DELETE |
| --- | --- |
| URL | {{baseUrl_api}}/admin/inventory |

Headers:

```
Authorization: Bearer {{adminToken}}
```

Body JSON (for POST/PUT):

```
{
  "ingredient_name": "Basil",
  "current_stock": 10,
  "unit": "g",
  "reorder_level": 3,
  "supplier": "Herbs & Co",
  "unit_cost": 0.5,
  "expiry_date": "2026-09-01"
}
```

Body: without body (for GET and DELETE).

### GET /api/v1/admin/analytics

Retrieve analytics data. Requires admin role.

| Method | GET |
|---|---|
| URL | {{baseUrl_api}}/admin/analytics |

Headers:

```
Authorization: Bearer {{adminToken}}
```

Body: without body.

## Recommended test order

1. POST /api/v1/auth/signup
2. POST /api/v1/auth/login — save accessToken
3. GET /api/v1/menu — copy a real dishId
4. POST /api/v1/cart/add with Authorization header
5. GET /api/v1/cart
6. POST /api/v1/checkout
7. GET /api/v1/orders
8. POST /api/v1/reservations
9. GET /api/v1/reservations
10. GET /api/v1/surveys/active — copy surveyId
11. POST /api/v1/surveys/:surveyId/respond
12. GET /ops/surveys (ops channel)
13. POST /ops/ingredients (ops channel)
14. GET/POST/PUT/DELETE /api/v1/admin/inventory — requires adminToken
15. GET /api/v1/admin/analytics — requires adminToken

## Important notes

If a protected route returns 401, it is usually because the Authorization header is missing, the token has expired, or the user does not exist.

If an admin route returns 403, the user exists but does not have the admin role.

If it returns 404, verify that you are using the correct base URL prefix (/ops or /api/v1) and that the ID exists in the database.

For production, replace the baseUrl variables with the configured domain or public IP.